Security Audit Report

Gno Sprint 2

v1.0

January 27, 2025

Table of Contents

Table of Contents 2
License 3
Disclaimer 4
Introduction 5
Purpose of This Report 5
Codebase Submitted for the Audit 5
Methodology 7
Functionality Overview 7
How to Read This Report 8
Code Quality Criteria 9
Summary of Findings 10
Detailed Findings 11
1. Non-determinism in pointer address logging causes inconsistent block production 11
2. Deleted objects are not correctly marked, causing memory leaks 11
3. Insufficient gas metering and unmetered operations lead to denial of service 12
4. Memory leaks in Stringer methods affect node resource consumption 14
5. Read-only execution is not enforced due to an uninitialized field 14
6. Unbounded cache growth allows memory exhaustion attacks 15
7. Improper ObjectID parsing causes integer overflows and non-determinism 15
8. Potential memory leak in convertTo during native-type conversions 16
9. Missing location attributes in SwitchClauseStmt 16
10. Inconsistent error handling in ParseMemPackage 17
11. Usage of panics instead of structured error handling 17
12. Redundant code leading to inefficiencies 18
13. Improper type usage causes unnecessary panics and inefficiencies 19
14. Usage of magic literals reduces maintainability 19
15. Outstanding TODO and XXX comments 20
16. Misleading comments and error messages 21
Appendix 22
1. Proof of concept for “Non-determinism in pointer address logging causes inconsistent block production” 22

License

THIS WORK IS LICENSED UNDER A CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE.

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUE ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

https://oaksecurity.io/
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by NewTendermint LLC to perform a security audit of Gno.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/gnolang/gno
Commit	`1b89166af37dd2ee63f5a30b81769532fd0bb1f7`
Scope	The audit focuses on key components of the GnoVM ecosystem, including storage mechanisms, non-deterministic behavior, and virtual machine (VM) validity. Specific areas of interest include: • Realm Storage

	○ Evaluation of functions and mechanisms related to realm updates and transaction finalization. This includes a review of the following files: ■ `gnovm/pkg/gnolang/ownership.go` ■ `gnovm/pkg/gnolang/realm.go` ■ `gnovm/pkg/gnolang/store.go` ● Non-Determinism ○ We focused on key language features and their mapping to the underlying Go implementation to find non deterministic paths, specifically reviewing: ■ `gnovm/pkg/gnolang/preprocess.go` ■ `gnovm/pkg/gnolang/values_string.go` ■ `gnovm/pkg/gnolang/machine.go` ■ `gnovm/pkg/gnolang/op_expressions.go` ● VM Validity ○ Assessment of the validity and correctness of virtual machine operations, particularly in: ■ `gnovm/pkg/gnolang/preprocess.go` ■ `gnovm/pkg/gnolang/go2gno.go`

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
    a. Race condition analysis
    b. Under-/overflow issues
    c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Gno is an interpreted and fully-deterministic implementation of the Go programming language, designed to build succinct and composable smart contracts.

The first blockchain to use it is gno.land, a Proof of Contribution-based chain, backed by a variation of the Tendermint consensus engine.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	**High**	The codebase in scope includes the interaction of multiple components, including the gno.land chain, the GnoVM, and Tendermint. The chain incorporates unique concepts and while smart contracts are written in Gno, a new language derived from Go.
Code readability and clarity	**Medium**	Numerous unresolved TODOs have been found in the codebase. Some function definitions spans a large codebase which makes less clear their functioning.
Level of documentation	**Medium**	The VM is not documented extensively.
Test coverage	**Medium-High**	Test coverage reported by `go test` for `gnovm/pkg` is `69.50%`.

Summary of Findings

No	Description	Severity	Status
1	Non-determinism in pointer address logging causes inconsistent block production	Critical	Reported
2	Deleted objects are not correctly marked, causing memory leaks	Critical	Reported
3	Insufficient gas metering and unmetered operations lead to denial of service	Critical	Reported
4	Memory leaks in `Stringer` methods affect node resource consumption	Major	Reported
5	Read-only execution is not enforced due to an uninitialized field	Major	Reported
6	Unbounded cache growth allows memory exhaustion attacks	Major	Reported
7	Improper `ObjectID` parsing causes integer overflows and non-determinism	Major	Reported
8	Potential memory leak in `convertTo` during native-type conversions	Minor	Reported
9	Missing location attributes in `SwitchClauseStmt`	Informational	Reported
10	Inconsistent error handling in `ParseMemPackage`	Informational	Reported
11	Usage of panics instead of structured error handling	Informational	Reported
12	Redundant code leading to inefficiencies	Informational	Reported
13	Improper type usage causes unnecessary panics and inefficiencies	Informational	Reported
14	Usage of magic literals reduces maintainability	Informational	Reported
15	Outstanding `TODO` and `XXX` comments	Informational	Reported
16	Misleading comments and error messages	Informational	Reported

Detailed Findings

1. Non-determinism in pointer address logging causes inconsistent block production

Severity: Critical

In `gnovm/pkg/gnolang/values_string.go:294-296`, the memory addresses for pointers are printed via `fmt.Sprintf("%p", tv)`.

However, this is problematic because Go assigns memory addresses randomly, resulting in non-deterministic output.

This issue is exacerbated when `ProtectedSprint` processes typed values extracted from stack traces, embedding them in error messages generated by the `doRecover` function during a panic. These error messages are stored in `Abci.Results` within the `runTx` method.

Consequently, if a machine experiences a panic, the resulting non-deterministic error messages may lead to inconsistent block production across nodes, potentially halting the chain.

A minimal example demonstrating this behavior is provided in the Appendix.

Recommendation

We recommend removing the `%p` format specifier to eliminate non-deterministic memory address logging.

Status: Reported

This audit was performed on an early version of the protocol.

2. Deleted objects are not correctly marked, causing memory leaks

Severity: Critical

In `gnovm/pkg/gnolang/ownership.go:293-299`, the `SetIsDeleted` method of the `ObjectInfo` struct is responsible for marking objects as deleted by updating the `isDeleted` field.

However, instead of modifying the `isDeleted` field, the method incorrectly updates the `isDirty` field, unintentionally replicating the behavior of the `SetIsDirty` method.

Consequently, objects will not be correctly flagged as deleted. This leads to memory leaks and a potential crash if the machine does not have available memory to continue allocating resources, causing a denial of service.

Additionally, the method does not log the update time, increasing the difficulty of tracking object state transitions.

Recommendation

We recommend correcting the `SetIsDeleted` method to update the `isDeleted` field properly. Additionally, we recommend ensuring the method logs the update timestamp to maintain accurate records of object state transitions.

Status: Reported

This audit was performed on an early version of the protocol.

3. Insufficient gas metering and unmetered operations lead to denial of service

Severity: Critical

The current implementation in multiple areas of the `gnovm` package exhibits issues related to gas metering and resource management, which could lead to system inefficiencies and potential denial of service vulnerabilities:

Fixed gas costs for dynamic operations

In `gnovm/pkg/gnolang/machine.go`, the `Run` method charges gas for each operation by calling `incrCPU` with a fixed number of cycles.

However, this fixed-cost approach does not reflect the variability in computational demand.

For example, the `OpSliceLit2` operation in `gnovm/pkg/gnolang/op_expressions.go:588` processes slice literals with explicit index assignments has a predetermined gas cost of 467 cycles.

The associated function iterates based on the provided user input. If a user supplies extremely large integer keys for the slice, the system attempts to allocate a slice of length equal to the maximum key plus one.

Without proper bounds or memory checks, this behavior could lead to excessive memory allocation and computational resource usage, potentially causing a denial of service.

Unmetered `Stringer` methods

In `gnovm/pkg/gnolang/values_string.go:81-87`, the `ProtectedString` method iterates over array elements to stringify them but does not leverage the virtual machine's gas meter.

The issue is that stringifying large arrays or deeply nested structures can be computationally expensive. Since these operations are unmetered, attackers could exploit them to exhaust resources, leading to potential denial of service attacks.

Unmetered recursion patterns

In `gnovm/pkg/gnolang/realm.go`, several methods heavily rely on recursion for traversing object hierarchies, both when descending into child objects and ascending into ancestors. Examples include `incRefCreatedDescendants`, `decRefDeletedDescendants`, `saveUnsavedObjectRecursively` for descending operations, and `markDirtyAncestors` for ascending operations.

Additionally, recursive patterns are present in methods such as `getChildObjects2` and `getSelfOrChildObjects`, which retrieve child objects from `Value` instances by traversing arrays, maps, slices, struct fields, and blocks.

These recursive operations are essential for managing object hierarchy traversal and modifications. However, since these operations are not gas-metered, the computational resources consumed are not accounted for.

Similarly, in `gnovm/pkg/gnolang/machine.go:807`, the `RunStatement` method is responsible for executing a statement. This method retrieves the source from the store, performs preprocessing, configures operations within the machine, and then executes the statement. While most operations in this process are gas-metered, the `Preprocess` phase is not.

Specifically, the `Preprocess` function in `gnovm/pkg/gnolang/preprocess.go:421` calls the `Transcribe` function, which internally executes large recursive operations. These recursive calls are not metered for gas consumption. In scenarios involving complex source inputs with deeply nested structures, these unmetered recursions could become computationally expensive, consuming significant resources and potentially impacting system performance.

Consequently, this omission results in inefficiency for the chain and exposes the system to potential denial of service attacks, as malicious actors could exploit these unmetered operations to overwhelm the system.

Recommendation

We recommend replacing the fixed gas cost model with another model that dynamically accounts for the complexity of each operation:

- **Introduce dynamic gas calculations** based on input size and the number of iterations required by an operation. Conduct load tests to evaluate the performance of the updated gas metering logic under varying input sizes.
- **Review and enforce gas metering** for all operations involving the `protectedStringer` interface.

- **Implement gas metering for recursive operations** by adding gas consumption checks at strategic points. Establish upper limits for recursion depth and object size to mitigate the risk of excessive resource usage.

Status: Reported

This audit was performed on an early version of the protocol.

4. Memory leaks in `Stringer` methods affect node resource consumption

Severity: Major

In `gnovm/pkg/gnolang/values_string.go:117-122`, the `ProtectedString` method processes slices and arrays by creating a new string slice with `ss := make([]string, sv.Length)`.

During this process, dynamic memory allocation occurs. However, the transaction allocator does not track the allocated memory, which could lead to a significant memory leak when large slices or arrays are stringified.

For example, stringifying a slice with a size equal to the transaction allocator's default limit (e.g., `500MB`) results in an additional `500MB` of untracked memory allocation. This untracked memory causes excessive RAM consumption, adversely affecting node performance and stability during high system loads.

Recommendation

We recommend modifying the implementation to track memory allocations. Additionally, we recommend implementing test cases to perform a comprehensive review of all internal `make` calls within the codebase, identifying and addressing instances where untracked memory allocations could lead to leaks.

Status: Reported

This audit was performed on an early version of the protocol.

5. Read-only execution is not enforced due to an uninitialized field

Severity: Major

In `gnovm/pkg/gnolang/realm.go:324-336`, the `FinalizeRealmTransaction` function is responsible for finalizing state changes after machine execution. It includes a condition based on the `ReadOnly` field that intends to prevent state modifications from occurring during read-only operations.

However, this condition is never applied because the machine's `ReadOnly` field is never initialized. As a result, the check always evaluates to `false`, causing the function to

consistently bypass read-only restrictions. This issue also occurs in explicitly read-only scenarios, such as `QueryEval`, where the `ReadOnly` field remains uninitialized.

Consequently, the `FinalizeRealmTransaction` function fails to enforce read-only semantics correctly, potentially leading to undefined behavior or consistent panic states.

Recommendation

We recommend ensuring that the `ReadOnly` field of the machine is properly initialized in all scenarios requiring read-only execution, such as queries.

Status: Reported

This audit was performed on an early version of the protocol.

6. Unbounded cache growth allows memory exhaustion attacks

Severity: Major

In `gnovm/pkg/gnolang/realm.go:80`, the `pkgIDFromPkgPathCache` serves as a shared cache for package paths. While it is initialized with a capacity of 100, this value merely defines the initial allocation and does not impose an actual limit.

Consequently, attackers who can generate arbitrary package paths are able to manipulate the cache size by repeatedly calling `PkgIDFromPkgPath` with unique package path values. Since no eviction policy exists, the cache will grow indefinitely, leading to excessive memory consumption and potential denial of service issues for nodes.

Recommendation

We recommend implementing a least recently used or size-based eviction policy to prevent unbounded growth of `pkgIDFromPkgPathCache`.

Status: Reported

This audit was performed on an early version of the protocol.

7. Improper `ObjectID` parsing causes integer overflows and non-determinism

Severity: Major

In `gnovm/pkg/gnolang/ownership.go:52`, the `UnmarshalAmino` method processes a string to instantiate an `ObjectID`, which consists of `PkgID` and `NewTime` fields delimited by a colon character. The first part of the string is hex-decoded for `PkgID`, while the second is parsed as an integer for `NewTime` using `strconv.Atoi` before being cast into `uint64`.

However, on 32-bit systems, `NewTime` values represented as `uint64` may overflow due to the size difference between 32-bit and 64-bit integers, causing non-deterministic behavior

across nodes. Additionally, negative inputs could result in underflows, increasing the risk of erroneous processing or exploitation.

Recommendation

We recommend replacing `strconv.Atoi` with `strconv.ParseUint(parts[1], 10, 64)` to ensure compatibility with 64-bit integers.

Status: Reported

This audit was performed on an early version of the protocol.

8. Potential memory leak in `convertTo` during native-type conversions

Severity: Minor

In `gnovm/pkg/gnolang/values_conversions.go:34-45`, the `convertTo` function processes type conversions for `NativeType` values. When both the source type (`tv.T`) and destination type (`t`) are `NativeType`, the memory allocation required for the conversion is not explicitly tracked.

For example, converting an `int8` value to `int64` requires an additional 7 bytes of memory. However, this allocation is not tracked.

Consequently, repeated conversions involving `NativeType` could lead to cumulative untracked memory usage, potentially causing memory leaks. Over time, this degrades node performance and stability, particularly under high-load conditions.

Recommendation

We recommend explicitly tracking all memory allocations during `NativeType` conversions using the `alloc` mechanism. Additionally, we recommend reviewing other functions that perform similar type-conversions and introducing test cases to detect and prevent untracked memory allocations.

Status: Reported

This audit was performed on an early version of the protocol.

9. Missing location attributes in `SwitchClauseStmt`

Severity: Informational

In `gnovm/pkg/gnolang/go2gno.go:950-978`, the `toClauses` function that processes switch clauses does not set the `Line` and `Column` attributes for `SwitchClauseStmt`. Other methods, such as `toDecls`, explicitly call `setLoc` to assign these values, but `toClauses` does not.

Consequently, the absence of `Line` and `Column` attributes leads to the following issues:

- **Debugging challenges**: errors within switch clauses cannot be traced back to their source location, increasing troubleshooting difficulty.
- **Inconsistent stack traces**: debugging tools may generate incomplete or misleading stack traces, reducing the reliability of error reports.
- **Tooling limitations**: development tools that rely on these attributes for analysis and automation may fail or produce incorrect results.

Recommendation

We recommend modifying the `toClauses` or `toSwitchClauseStmt` functions to call `setLoc`.

Status: Reported

This audit was performed on an early version of the protocol.

10. Inconsistent error handling in `ParseMemPackage`

Severity: Informational

In `gnovm/pkg/gnolang/nodes.go:1278`, the `ParseMemPackage` function panics when the file being processed refers to a package different from the current one. However, this behavior is inconsistent with the function's general error-handling approach.

For example, in line `1274`, parsing errors are appended to the `errs` accumulator variable rather than triggering an immediate panic. These accumulated errors are then used to generate a comprehensive error message in line `1286`.

Recommendation

We recommend appending the error message to the `errs` list instead of panicking immediately to maintain consistency in error handling.

Status: Reported

This audit was performed on an early version of the protocol.

11. Usage of panics instead of structured error handling

Severity: Informational

The codebase heavily relies on panics instead of returning error values. While panics may be appropriate in certain scenarios, it is usually a trade-off:

- **Missed error handling**: panics are not subject to type-checking, so it is easier to overlook error-handling mechanisms.
- **Unrecoverable failures**: if a panic signals a recoverable error and is not properly handled, the program may terminate unnecessarily.
- **Reduced readability and debugging complexity**: codebases that rely on panics are harder to follow and debug, increasing maintenance overhead.
- **Performance overhead**: unwinding the stack decreases program performance.

Consequently, relying on panics as the primary error-handling mechanism increases the difficulty of ensuring codebase safety and maintainability.

Recommendation

We recommend returning errors with meaningful error messages rather than using panics. This approach improves code maintainability, debugging efficiency, and overall developer experience.

Status: Reported

This audit was performed on an early version of the protocol.

12. Redundant code leading to inefficiencies

Severity: Informational

A few instances of redundant code have been identified in the codebase:

- In `gnovm/pkg/gnolang/machine.go:2087`, the `MustLastCallFrame(1)` function is called to retrieve the last call frame, but this value is later discarded in line `2093` by `PopUntilLastCallFrame`. This results in an unnecessary function call and can be removed.
- In `gnovm/pkg/gnolang/machine.go:1556`, the `PeekStmt` function is implemented to accept an arbitrary parameter `offset`. However, all three usages of this function pass `1` as the parameter, causing it to be redundant. Since this flexibility is never utilized, the function can be replaced with `PeekStmt1`.

Recommendation

We recommend applying the recommendations in the abovementioned instances.

Status: Reported

This audit was performed on an early version of the protocol.

13. Improper type usage causes unnecessary panics and inefficiencies

Severity: Informational

A few instances of improper type usage have been identified in the codebase:

- In `gnovm/pkg/gnolang/machine.go:1938`, the `lastCallFrame` function accepts a parameter `n` of type `int`, but it does not handle negative values correctly. Instead of panicking when `n` is zero, the function could simply return the frame at the top of the stack. The condition in line `1945` should be replaced with `n == 0` to allow this behavior.
- Similarly, the `X_callerAt` function also accepts an `int` parameter `n`, but it panics when `n <= 0`. Negative values could be prevented by using an unsigned integer type, while zero could be interpreted as a valid input rather than causing a panic.
- In `gnovm/pkg/gnolang/nodes.go:121-122`, the `Line` and `Column` fields in the `Location` structure are declared as signed integers, despite negative values being neither required nor allowed. Using an unsigned type would better reflect their intended use.

Recommendation

We recommend using unsigned integer types when negative values are not needed. Additionally, function parameters with zero values should be interpreted as valid inputs when appropriate.

Status: Reported

This audit was performed on an early version of the protocol.

14. Usage of magic literals reduces maintainability

Severity: Informational

Throughout the codebase, hard-coded number literals without context or a description are used. Using such "magic numbers" goes against best practices as they reduce code readability and maintenance as developers are unable to easily understand their use and may make inconsistent changes across the codebase.

Instances of magic literals include:

- In `gnovm/pkg/gnolang/nodes.go:1268-1269`, file names are validated to contain the Gno extension and not being marked as test files. In particular, string literals "`.gno`", "`_test.gno`" and "`_filetest.gno`" are used to define the corresponding conditions.

- Similarly, in `gnovm/pkg/gnolang/go2gno.go:596-598`, equivalent code with the same string literals is used. These literals appear frequently across the codebase, with "`.gno`" alone appearing 20 times in 15 different files.
- In `gnovm/stdlibs/std/native.go:166`, the expression `m.NumFrames()-2` is used as an index in the frames array, but it is unclear why the number 2 is specifically required to determine the caller's frame.

Consequently, using string literals directly severely decreases maintainability and increases the likelihood of introducing bugs due to typographical mistakes.

Recommendation

We recommend declaring named constants for important string values and documenting those that are part of user-facing APIs. Additionally, canonical predicates should be defined for string comparisons to ensure consistency.

Status: Reported

This audit was performed on an early version of the protocol.

15. Outstanding `TODO` and `XXX` comments

Severity: Informational

The codebase contains numerous `TODO` and `XXX` comments. This approach to documenting issues and outstanding work has a few disadvantages:

- `TODO` comments often become stale compared to properly tracked issues, leading to a lack of progress toward resolution.
- Information in comments must be maintained and updated. Otherwise, it may be misleading for future developers or reviewers.
- These comments highlight weak points in the codebase, which could be exploited by malicious actors since the code is publicly accessible.

A non-exhaustive list of discovered `TODO` comments includes:

- `gnovm/pkg/gnolang/machine.go:1181`
- `gnovm/pkg/gnolang/op_expressions.go:71,184`
- `gnovm/pkg/gnolang/preprocess.go:4831-4835`

The total number of such comments exceeds 200 across the codebase.

Recommendation

We recommend addressing the most pressing `TODO` and `XXX` comments while moving the remaining ones into a structured issue-tracking system.

Status: Reported

This audit was performed on an early version of the protocol.

16. Misleading comments and error messages

Severity: Informational

In `gnovm/pkg/gnolang/transcribe.go:132`, the `Transcribe` function panics with the message "*`Transcribe()` expects a non-pointer concrete `Node` struct*".

However, this panic is triggered when `reflect.TypeOf(n).Kind() != reflect.Ptr`, contradicting the function's inline comment, which states: "*`n` MUST be a pointer*".

Recommendation

We recommend correcting the error message to accurately reflect the function's behavior and keeping the inline documentation up to date.

Status: Reported

This audit was performed on an early version of the protocol.

Appendix

1. **Proof of concept for "Non-determinism in pointer address logging causes inconsistent block production"**

The following code illustrates how memory addresses vary when `Stringer` methods are invoked on objects with pointer references.

```
package main

type MyStruct struct {
	Value int
	Next  *MyStruct // Pointer to another MyStruct (or itself)
}

func Test() {
	structs := []*MyStruct{
		{Value: 42},  // First object
		{Value: 42},  // Second object, same value as the first
		{Value: 100}, // Third object, unique value
	}

	// Create recursive references
	structs[0].Next = structs[0]
	structs[1].Next = structs[0]
	structs[2].Next = structs[2]

	// Print memory addresses and values
	for i, s := range structs {
		println("Struct", i, "Address:", s, "Value:", s.Value, "Next
Address:", s.Next)
	}
}
```

Running the `Test` function on multiple machines generates different pointer addresses, such as:

- Struct 0 Address: 0x1400d3ed8f0 Value: 42 Next Address: 0x1400d3ed8f0
- Struct 1 Address: 0x1400d3ed9f0 Value: 42 Next Address: 0x1400d3ed8f0
- Struct 2 Address: 0x1400d428840 Value: 100 Next Address: 0x1400d428840

While this randomness does not affect `Print` or `Println` operations (as their output is discarded), invoking `Stringer` during persistent operations (like panics and stacktrace) can lead to non-deterministic behavior.